

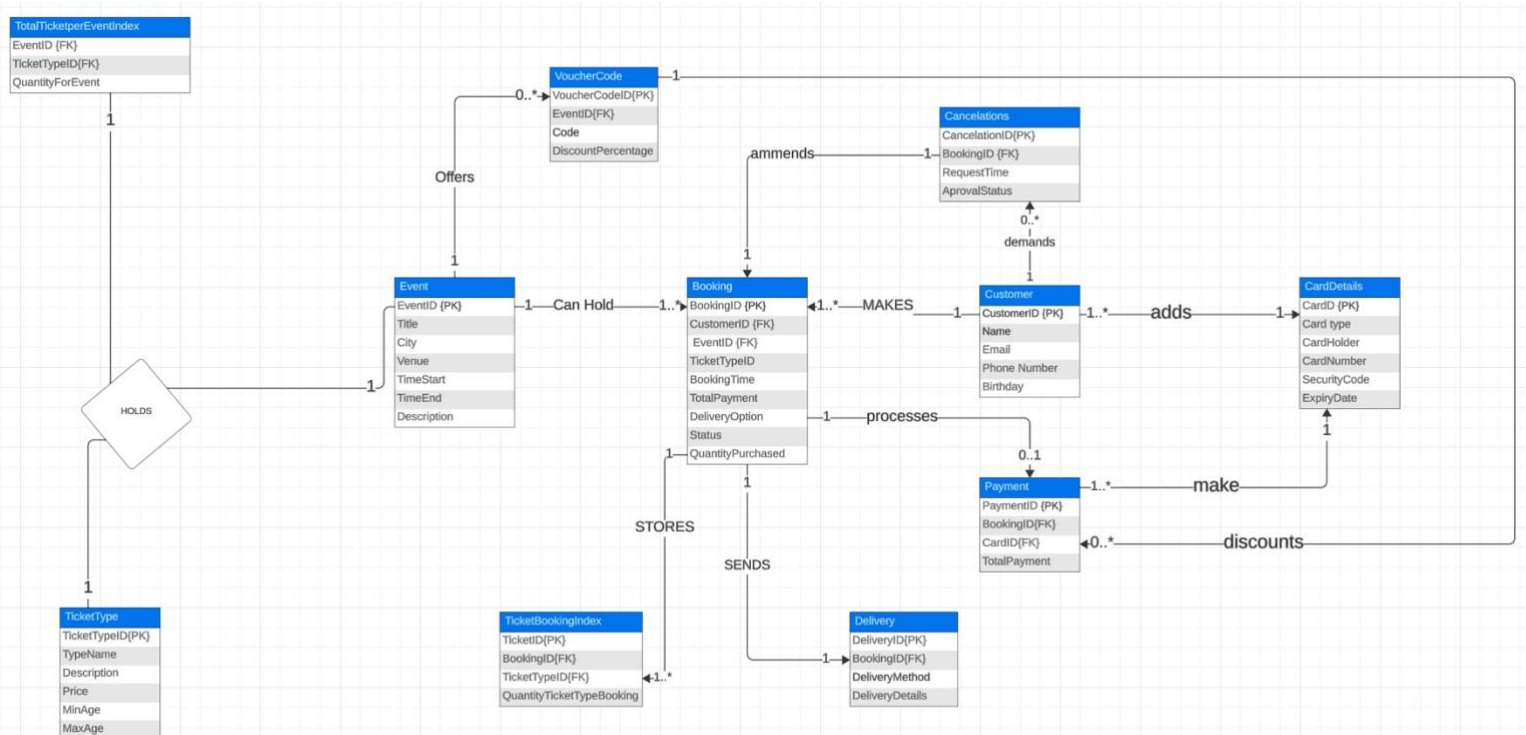
Ticket Design

I am writing a report detailing the design of the Ticket Booking Database system, which comprises two main parts; the Entity Relationship Diagram using UML notation and Logical model design. Image of my ERD:

Systematic Description

Ticket Design

I am writing a report detailing the design of the Ticket Booking Database system, which comprises two main parts; the Entity Relationship Diagram using UML notation, and Logical model design. Image of my ERD:



Systematic Description

My database consist of 11 entities these include; Customer, Booking, Event, Payment, CardDetails, Cancellation, TicketBookingIndex, TotalTicketperEventIndex, TicketType, VoucherCode, Delivery. I will break down each entity and the rationale below.

Booking Table Entity
and attributes

The Booking Entity is one of the main parts of the database, it has a direct relationship with the customer and event.

Within the Booking table, there are 9 attributes that make up the whole entity; BookingID, CustomerID, EventID, BookingTime, TotalPayment, DeliveryOption, Status, TicketTypeID, QuantityPurchased.

Keys

The Primary Key that is used for this entity is: **BookingID**. I used this name for the ID as it has the same name as the table. This shows it is unique to that table. The first Foreign Key that is used for this entity is: **CustomerID**. I used this as the first foreign key, because a booking needs to have a customer attached, so this references the **Customer (CustomerID)** who made each booking.

The second Foreign Key that is used for this entity is: **EventID**. I used this as a foreign key, because each booking has to be attached to an event for which a customer wants to attend, this references **Events(EventID)**.

The Third Foreign Key that is used for this entity is: **TicketTypeID**. I used this as a foreign key, because this references **TicketType(TicketTypeID)**, the description of the type of ticket being booked. **Cancellation Table** Entity and attributes

This entity holds the data for customers who want to cancel their bookings. It will use the references from Booking ID such as whether the request time is after the start of the booking, to determine whether they can receive a refund.

The four attributes in this entity are the CancellationID, BookingID, RequestTime, ApprovalStatus. The Request Time is the time that the user requested the refund, and ApprovalStatus holds data on whether the cancellation was approved or not.

Keys

The Primary Key that is used for this table: **CancellationID**. I used this as the primary key, because there needed to be a unique name for each booking cancellation.

The Foreign Key that is used for this table: **BookingID**. This references the **Booking(BookingID)**, that is attached to each cancellation.

Card Details Table

Entity and attributes

This table holds the data for the Card Details for each customer, so it can be used to process a payment.

This entity holds 6 attributes that all relate to important details needed to process a payment, these include; CardID, CardType, CardHolder, CardNumber, SecurityCode, ExpiryDate. **Primary keys**

The Primary Key used is: **CardID**, this will be the same number as the CustomerID as I want it to be easy to be able to relate the customer to their card.

There was no foreign key used to make this table, as the primary key is identical to the each CustomerID. **Customer Table**

Entity and attributes

The Customer Entity records important data for each customer making a booking.

The customer entity holds 5 attributes which are; CustomerID (Primary Key), Name, Email, Phone, Birthdate. Some of the attributes are key information needed to communicate with the customers; name, email, and phone. The birthday attribute was added, so we can make sure a customer has the correct age requirement for the ticket they are purchasing.

Primary key

The Primary Key that was used: **CustomerID**. This is important in being able to uniquely identify each customer, in case any have the same name. **Delivery**

Table Entity and attributes

The Delivery entity is used, to enable customers to choose a delivery option for their ticket once the booking has been recorded. This is either in the form of emailing the ticket, or posting it to the address recorded in the DeliveryDetails. The 4 attributes of the table are; DeliveryID, BookingID, DeliveryMethod, DeliveryDetails. The delivery details will record the address or email that the user wants to send their ticket sent to, and the deliveryMethod will be store “email” or “address” to let us know what the customer wants. The BookingID will make sure we reference booking table.

Primary keys

The Primary Key used: **DeliveryID**, I wanted a clear name for this key and it needed its own so we can reference each delivery separately for the booking

The Foreign Key used : **BookingID** references **Booking(BookingID)**, like this we can reference each booking to make sure that every booking has picked an option for delivery that is needed. **Events Table** Entity and attributes

The event table will hold key data about each event that we will hold

This entity contains 7 attributes; EventID, Title, City, Venue, StartTime, EndTime, Description. The Title will be the Name of the Event, City will be the city the Event is going to be in, and the venue will be their location in the city it will be held. StartTime and EndTime will hold dateTime for the start of event and the end of event.

Primary keys

The Primary Key that was used was **EventID**, as each event will need a unique identify just in case the same event is held in different location so there is no confusion.

TotalTicketperEventIndex Table

Entity and attributes

The Event ticket will be used as an index to help identify, An Event to Which ticket types they want to sell and the quantity of each type of ticket they want to sell. The three attributes will be EventID, TicketTypeID, QuantityForEvent. They will be used to reference event to ticket type and output a quantity for ticket type.

keys

For this entity I used a Composite Primary Key which is slightly different: (**EventID**, **TicketTypeID**)

The first Foreign Key that was used is: **EventID** references **Events(EventID)**

The second Foreign Key used is: **TicketTypeID** references

TicketType(TicketTypeID)

I thought a composition primary key was the best solution as they only stores data based on two entities. **Payment Table**

Entity and attributes

The payment entity records each payment for booking and record the total that need or has been paid.

The 4 attributes I have chosen is; PaymentID, BookingID, CardID, TotalPayment

Primary keys

The primary key that was used is : **PaymentID**

I used two foreign keys which are: **BookingID** references **Booking(BookingID)**, and **CardID** references **CardDetails(CardID)** **TicketBookingIndex Table** Entity and attributes

This entry hold data how ticket each customer has purchased of each type. The attributes that have been used are; TicketID, BookingID, TicketTypeID, Quantity. This is meant to act little an index for a booking an and all the different ticket a customer has wanted to purchase.

Keys

Primary Key that was used is: **TicketID**

The first Foreign Keys: **BookingID** references **Booking(BookingID)** and the second foreign key is **TicketTypeID** references **TicketType(TicketTypeID)** **TicketType**

Table Entity and attributes

This world all the different ticket type information and the requirement to purchase the tickets.

The entity holds 6 attributes; TicketTypeID, TypeName, Description, Price, MinAge, MaxAge.

MaxAge and MinAge are used adding requirement needed for purchasing a ticket whether customer is old or young enough, and the rest is just needed data for a ticket such as name and price.

Primary keys

The Primary Key that is used is : **TicketTypeID** and the Foreign Key used is:

EventID references **Events(EventID)**. **VoucherCode Table**

Entity and attributes

Voucher code entity holds data for; each voucher, which event they can be used at, and how much of a discount you can receive.

There are 4 entity attributes; VoucherCodeID, EventID, Code, DiscountPercentage.

Primary keys

Primary Key: **VoucherCodeID**

Foreign Key: **EventID** references **Events(EventID)**

Relationships and cardinality of relationships

The Customer has a (1)-> (1..*) relationship with Booking

The Event has a (1)-> (1..*) relationship with Booking
 The Booking has a (1)-> (1) relationship with Delivery
 Cancellation has a (1)-> (1) relationship with Booking
 Customer has a (1)->(0..*) relationship with cancellation
 Booking has a (1)-> (0..*) relationship with Payment
 Booking has a (1)-> (1..*) relationship with TicketBookingIndex
 CardDetails has a (1..*)-> (1) relationship with Payment
 Event has a (1)-> (0..*) relationship with VoucherCode
 Vouchercode has (0..*)->(1) relationship with payment
 Customer has (1..*)->(1) relationship with CardDetails

Complexities

When it comes to the tertiary relationship, where an event and the ticketType uses TotalTicketperEventIndex well as an index. My justification for using a complex relationship in this situation is because an event and the ticket it has are intertwined, and both will be organized alongside each other.

Relational Model:

1. **Booking:**
 - Attributes: BookingID (PK), CustomerID (FK), EventID (FK), BookingTime, TotalPayment, DeliveryOption, Status, TicketTypeID (FK), QuantityPurchased.
2. **Cancellation:** ○ Attributes: CancellationID (PK), BookingID (FK), RequestTime, ApprovalStatus.
3. **CardDetails:**
 - Attributes: CardID (PK), CardType, CardHolder, CardNumber, SecurityCode, ExpiryDate.
4. **Customer:** ○ Attributes: CustomerID (PK), Name, Email, Phone, Birthdate.
5. **Delivery:** ○ Attributes: DeliveryID (PK), BookingID (FK), DeliveryMethod, DeliveryDetails.
6. **Events:**
 - Attributes: EventID (PK), Title, City, Venue, StartTime, EndTime, Description.
7. **TotalTicketperEventIndex:**
 - Attributes: EventID (FK), TicketTypeID (FK), QuantityForEvent.
8. **Payment:**
 - Attributes: PaymentID (PK), BookingID (FK), CardID (FK), TotalPayment.
9. **TicketBookingIndex:**
 - Attributes: TicketID (PK), BookingID (FK), TicketTypeID (FK), QuantityTicketTypeBooking.
10. **TicketType:**
 - Attributes: TicketTypeID (PK), TypeName, Description, Price, MinAge, MaxAge.
11. **VoucherCode:**
 - Attributes: VoucherCodeID (PK), EventID (FK), Code, DiscountPercentage.